

PER2-PROTAC: Data exploration and statistical testing

Tom Smith

2026-01-30

Here, we explore the protein-level quantitative data and perform statistical testing to identify phosphorylation changes upon PROTAC treatment.

Setup

Load required packages

Define directory for plots

```
plots_dir <- here('results/plots')
if(!dir.exists(plots_dir)){
  dir.create(plots_dir, recursive = TRUE)
}
```

Read in the data

```
output_dir <- here('notebooks/output')

qfs <- readRDS(here(output_dir, 'PROTAC_qfs.rds'))

long_format_protein <- readRDS(
  here(output_dir, 'PROTAC_long_format_protein.rds'))

long_long_format_phosphopeptide_norm_to_protein <- readRDS(
  here(output_dir, 'PROTAC_long_long_format_phosphopeptide_norm_to_protein.rds'))
```

Data exploration

PCA plots

Below, we make PCA plots for the protein and phosphopeptide data. At the protein level, DMSO and PROTAC separate on PC1 (30% of variance), with two DMSO-2 and PROTAC-3 relatively close together. For phosphopeptides, the conditions separate very well (PC1; ~40% variance explained)

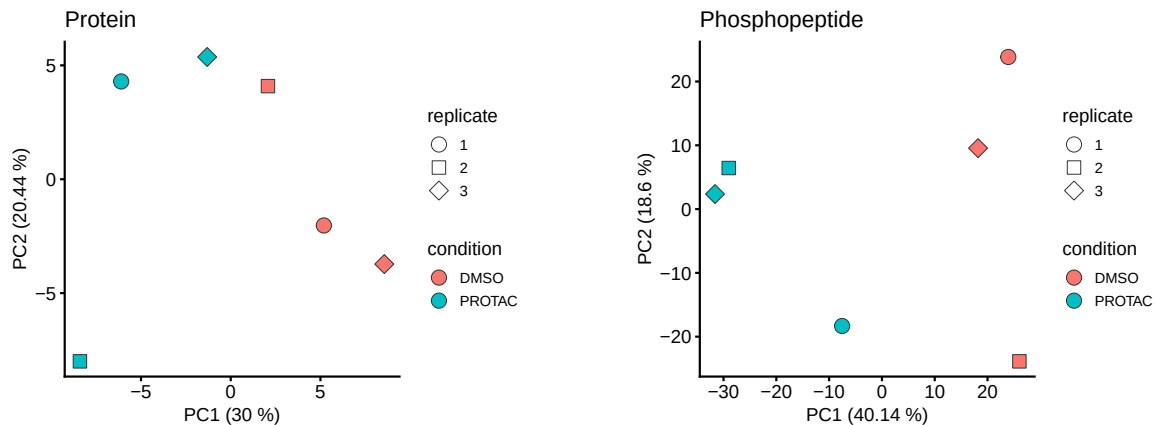
```
for(qf in names(qfs)){
  if(qf=='total'){
    assay_name <- 'protein'
  }
}
```

```

    feature <- 'Protein'
  } else{
    assay_name <- 'phosphopeptide'
    feature <- 'Phosphopeptide'
  }

  print(plot_pca(
    qfs[[qf]], i=assay_name, colour_by = 'condition', shape='replicate') +
    ggtitle(feature))
}

```



Statistical testing of protein abundance changes between conditions

Protein

First, we'll perform statistical testing to compare the conditions (PROTAC vs DMSO) for total protein abundance

```

## Create design matrix for the linear model
condition <- factor(qfs$total[['protein']]$condition, levels=c('DMSO', 'PROTAC'))
m_design <- model.matrix(~condition)

## Fit linear model using the design matrix and desired contrasts
fit_model <- lmFit(object = assay(qfs$total[['protein']]), design = m_design)

## Update the model using the limma eBayes algorithm
final_model <- eBayes(fit = fit_model,
                      trend = TRUE,
                      robust = TRUE)

## Add TREAT to final model to perform fold-change thresholding for contrasts
# require |FC| significantly over 1.25x
final_model_treat <- treat(fit_model, fc=1.25, trend=TRUE, robust=TRUE)

## QC plot to assess if there is a trend between abundance and variance, as expected

```

```

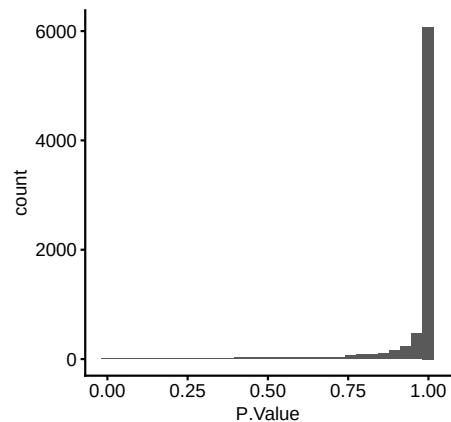
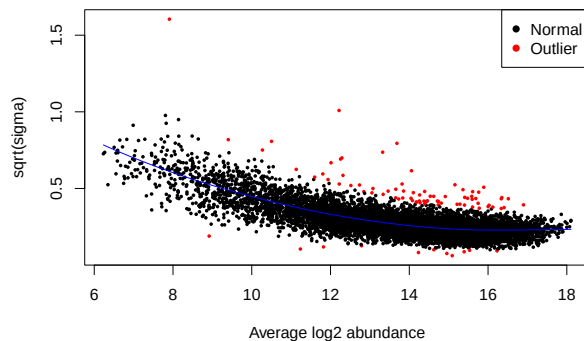
plotSA(fit = final_model,
       cex = 0.5,
       xlab = "Average log2 abundance")

## Format results
prot_limma_results <- topTreat(
  fit = final_model_treat,
  coef = 'conditionPROTAC',
  adjust.method = "BH", # Method for multiple hypothesis testing
  number = Inf) %>% # Print results for all proteins
  tibble::rownames_to_column("Protein") %>%
  merge(data.frame(rowData(qfs$total)[['protein']] ),
        by.x='Protein', by.y='row.names', all.x=TRUE) %>%
  arrange(P.Value)

## Check distribution of p-values. Because we are using treat, we don't expect a
## uniform distribution under the null, but rather a peak near 1.
p <- prot_limma_results %>%
  as_tibble() %>%
  ggplot(aes(x = P.Value)) +
  geom_histogram() +
  theme_biomasslmb()

print(p)

```



How many proteins have a significant change in abundance

```

table(prot_limma_results$adj.P.Val<0.05,
      ifelse(prot_limma_results$logFC>0, 'Increased', 'Decreased')) %>%
  knitr::kable()

```

	Decreased	Increased
FALSE	3602	4125
TRUE	1	1

```
prot_limma_results %>%
  filter(adj.P.Val<0.05) %>%
  select(Protein, Gene.Names.First, logFC, adj.P.Val) %>%
  knitr::kable()
```

Protein	Gene.Names.First	logFC	adj.P.Val
O00469	PLOD2	-0.8429726	3.20e-06
Q16850	CYP51A1	0.7856152	4.21e-05

OK, so very few proteins have a significant change in abundance!

Phosphopeptide statistical testing

Next, we perform statistical testing for the phosphopeptide data, comparing PROTAC vs DMSO while accounting for the matched protein abundance measurements.

```
# Create data matrix for modelling
to_model_phospho <- long_long_format_phosphopeptide_norm_to_protein %>%
  filter(type!='phosphopeptide_prot_norm') %>%
  select(prot_phos, condition, replicate, type, abundance) %>%
  pivot_wider(names_from = c(condition, replicate, type), values_from=abundance) %>%
  tibble::column_to_rownames('prot_phos')

# Create design matrix for the linear model
exp_cond_phospho <- data.frame(sample=colnames(to_model_phospho)) %>%
  separate(sample, into=c('condition', 'replicate', 'type'), remove = FALSE)

condition <- factor(exp_cond_phospho$condition, levels=c('DMSO', 'PROTAC'))

type <- factor(exp_cond_phospho$type, levels=c('protein', 'phosphopeptide'))

m_design_phospho <- model.matrix(~ condition * type)

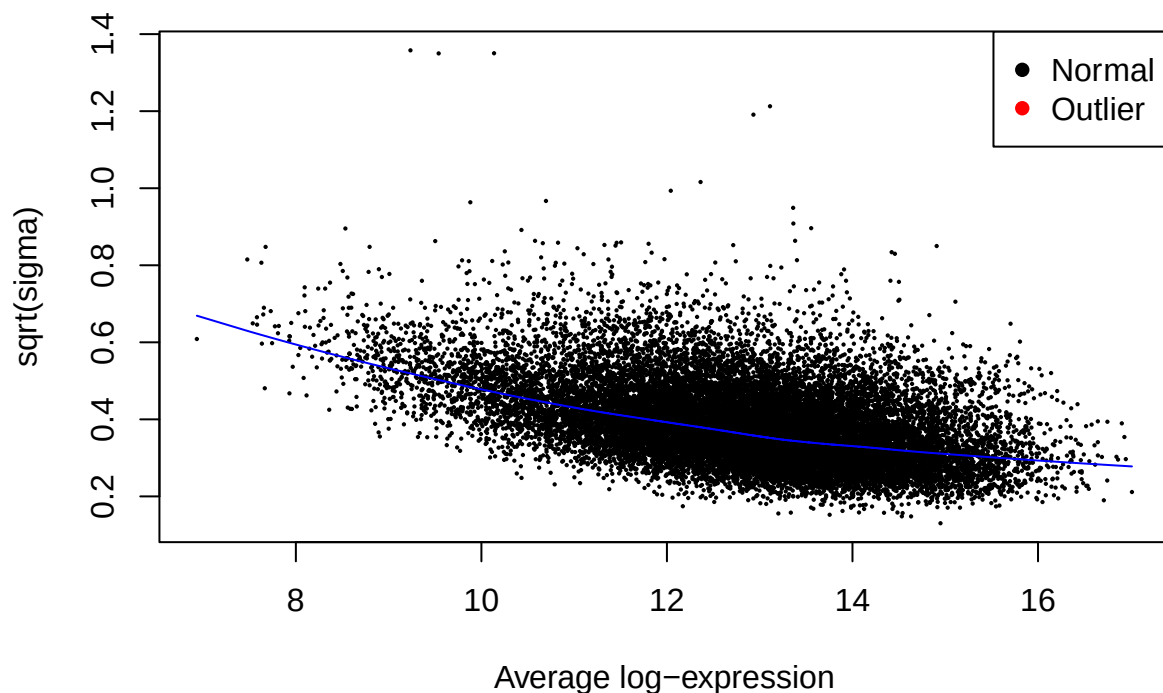
# same ID for phospho + total from the same sample
block <- interaction(condition, exp_cond_phospho$replicate)

# Estimate the correlation between measurements from the same block
corfit <- duplicateCorrelation(as.matrix(to_model_phospho),
                              m_design_phospho, block=block)

# Fit the linear model with blocking and correlation information
limma_fit_phospho <- lmFit(
  as.matrix(to_model_phospho), m_design_phospho, block=block, correlation=corfit$consensus)

# Update the model using the limma eBayes algorithm
limma_fit_phospho_eb <- eBayes(limma_fit_phospho, trend=TRUE, robust=TRUE)

# QC plot to assess if there is a trend between abundance and variance, as expected,
# and if the limma 'trend' captures this
plotSA(limma_fit_phospho_eb)
```



```
# Format results
phospho_limma_results <- topTable(limma_fit_phospho_eb, coef = 'conditionPROTAC:typephosphopeptide',
  adjust.method = "BH",      # Method for multiple hypothesis testing
  number = Inf) %>%         # Print results for all proteins
  tibble::rownames_to_column("phosphosite") %>%
  merge(rowData(qfs$phospho)[['phosphopeptide']][
    ,c('Entry.Name', 'Protein.names.y', 'Gene.Names', 'Gene.Names.First', 'site_seq')],
    by.x='phosphosite', by.y='row.names', all.x=TRUE) %>%
  data.frame() %>%
  separate(phosphosite, into=c('UniprotID', 'phospho_pos'), sep='_', remove=FALSE)
```

Save results

```
saveRDS(phospho_limma_results, here(output_dir, 'PROTAC_phospho_limma_results.rds'))
```

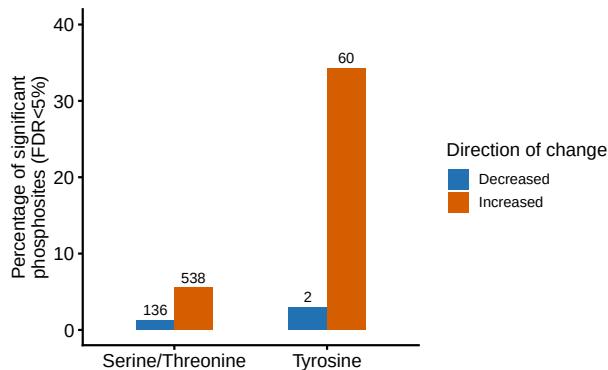
How many phosphosites have a significant change in abundance?

Below, we summarise how many phosphosites have a significant change in abundance (FDR<5%)

```
table(ifelse(phospho_limma_results$adj.P.Val<0.05, 'Significant', 'Not significant'),
  ifelse(phospho_limma_results$logFC>0, 'Increased', 'Decreased')) %>%
  knitr::kable()
```

	Decreased	Increased
Not significant	10899	9355
Significant	138	598

```
phospho_limma_results %>%
  mutate(type=ifelse(grepl('Y', phospho_pos), 'Tyrosine', 'Serine/Threonine')) %>%
  mutate(sig=adj.P.Val<0.05) %>%
  group_by(sig, type, direction=ifelse(logFC>0, 'Increased', 'Decreased')) %>%
  tally() %>%
  group_by(type, direction) %>%
  mutate(perc=100*n/sum(n)) %>%
  filter(sig) %>%
  ungroup() %>%
  complete(direction, type, fill=list(n=0)) %>%
  ggplot(aes(type, perc, fill=direction)) +
  geom_col(position='dodge', width=0.5) +
  theme_biomasslmb() +
  labs(x='', y='Percentage of significant\nphosphosites (FDR<5%)', fill='Direction of change') +
  scale_fill_manual(values=get_cat_palette(2)) +
  geom_text(aes(label=n), position=position_dodge(width=0.5), vjust=-0.5, size=4) +
  ylim(0, 40) +
  theme(strip.background = element_blank())
```



Below, we make a volcano plot for the statistical test results

```
# highlight top 10 significant proteins per genotype
phospho_limma_results %>%
  group_by(logFC>0) %>%
  filter(adj.P.Val<0.05) %>%
  slice_min(P.Value, n=10) -> highlight

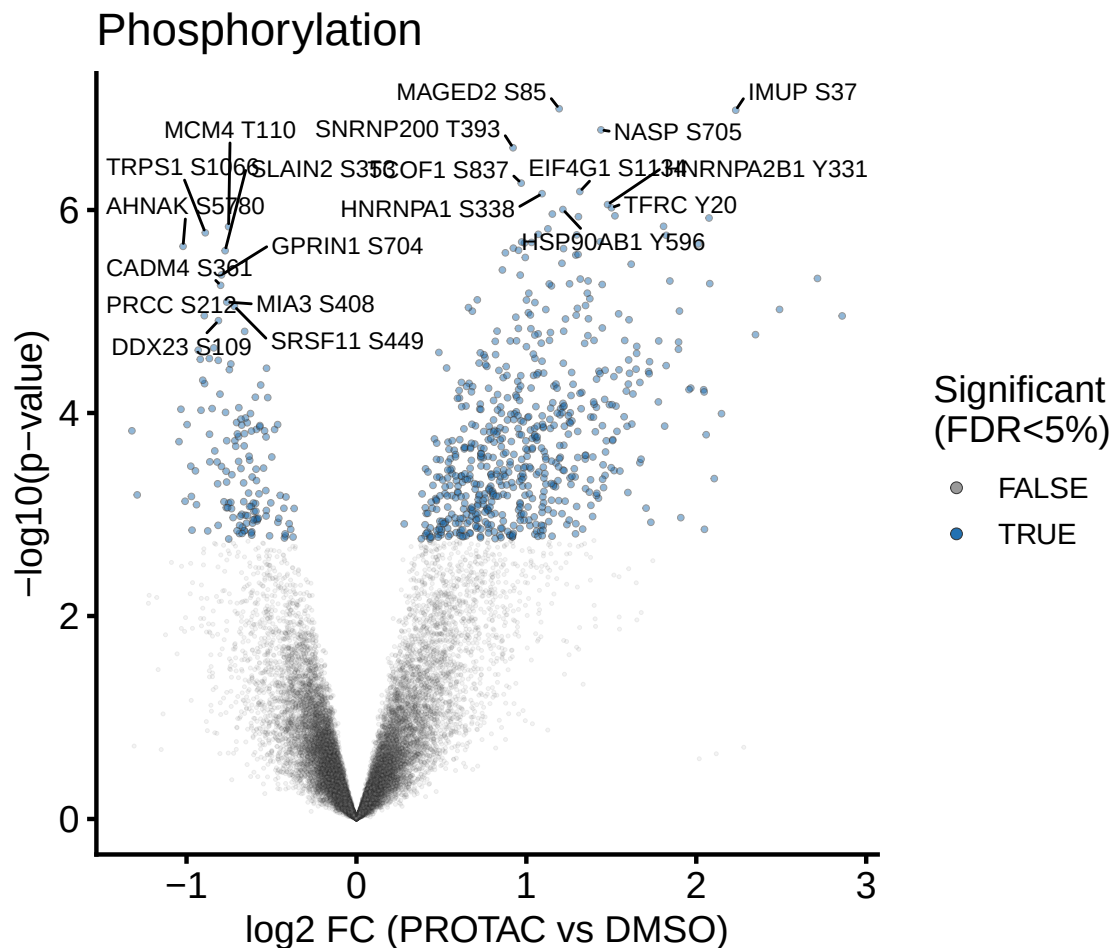
phospho_limma_results %>%
  mutate(significant=adj.P.Val<0.05) %>%
  ggplot(aes(logFC, -log10(P.Value))) +
  geom_point(aes(fill=significant,
                 alpha=significant, size=significant),
            colour='grey20', stroke=0.25, shape=21) +
  theme_biomasslmb() +
```

```

scale_fill_manual(values=c('grey60', get_cat_palette(1)),
                  name='Significant\n(FDR<5%)') +
scale_size_manual(values=c(0.5, 1), guide='none') +
scale_alpha_manual(values=c(0.1, 0.5), guide='none') +
labs(x='log2 FC (PROTAC vs DMSO)', y='-\log10(p-value)') +
theme(strip.background = element_blank()) +
geom_text_repel(
  data=highlight,
  aes(label=paste(Gene.Names.First, gsub('.*_', '', phosphosite))),
  max.overlaps = 50, size=3, min.segment.length =0, colour='black', force = 10) +
ggtitle('Phosphorylation') -> p

print(p)

```



Plot changes in phosphorylation for AKT1S1

Below, we plot the changes in phosphorylation for AKT1S1, which has multiple significantly changing sites

```
p_values <- phospho_limma_results %>%
  filter(Gene.Names.First=='AKT1S1') %>%
  mutate(ptm_name=phosphosite)

to_plot <- long_long_format_phosphopeptide_norm_to_protein %>%
  filter(Gene.Names.First=='AKT1S1', type=='phosphopeptide_prot_norm')

DMSO_abundances <- to_plot %>%
  filter(condition=='DMSO') %>%
  group_by(prot_phos) %>%
  summarise(abundance=mean(abundance))

to_plot <- to_plot %>%
  merge(DMSO_abundances, by='prot_phos', suffix=c('_', '_DMSO')) %>%
  mutate(abundance_diff=abundance - abundance_DMSO) %>%
  group_by(condition, prot_phos) %>%
  mutate(mean_abundance_diff=mean(abundance_diff)) %>%
  arrange(desc(mean_abundance_diff)) %>%
  mutate(ptm_name_facet=gsub(';', ' ', '\n', ptm_name))

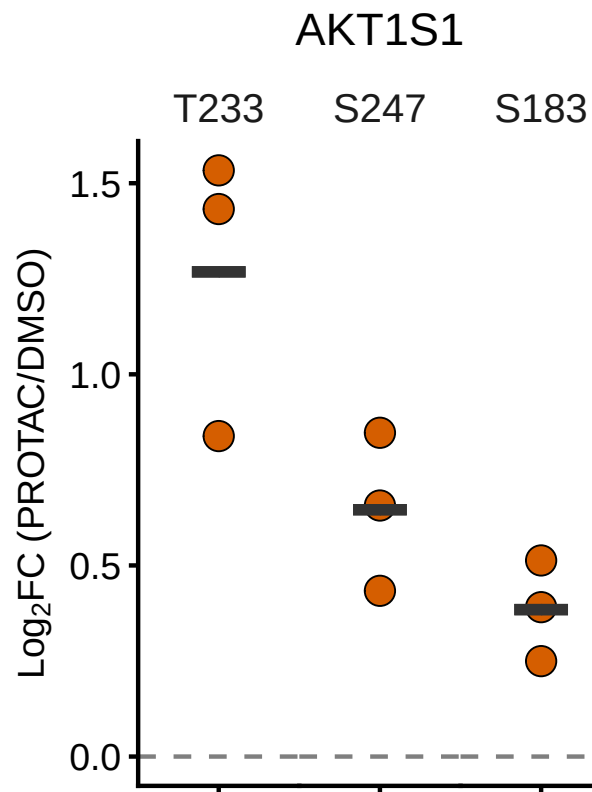
to_plot$ptm_name_facet <- factor(to_plot$ptm_name_facet,
                                levels=unique(to_plot$ptm_name_facet))

p_values <- phospho_limma_results %>%
  filter(Gene.Names.First=='AKT1S1')

sig_sites <- p_values %>%
  filter(adj.P.Val<0.1) %>%
  pull(phospho_pos)

p <- to_plot %>%
  filter(ptm_name %in% sig_sites) %>%
  filter(condition!='DMSO') %>%
  ggplot(aes(ptm_name, abundance_diff)) +
  geom_hline(yintercept = 0, colour='grey50', linetype='dashed') +
  geom_point(colour='black', stroke=0.5, shape=21,
             fill=get_cat_palette(2)[2], size=5) +
  stat_summary(fun.min=mean, fun.max=mean, geom='errorbar',
              linewidth=2, width=0.4, colour='grey20') +
  theme_biomasslmb(aspect_square = FALSE) +
  labs(x='', y=expression(Log[2]*FC~"(PROTAC/DMSO)")) +
  theme(axis.text.x=element_text(angle=45, hjust=1, vjust=1)) +
  facet_grid(.~ptm_name_facet, scales='free_x') +
  theme(strip.background = element_blank(),
        panel.spacing = unit(0, 'mm'),
        axis.text.x = element_blank(),
        aspect.ratio=4,
        strip.text.x=element_text(vjust=1, hjust=0),
        plot.title = element_text(hjust=0.5)) +
  ggtitle('AKT1S1')
```

```
p1 <- p + theme(strip.text.x=element_text(vjust=0.5, hjust=0.5, size=15))
print(p1)
```



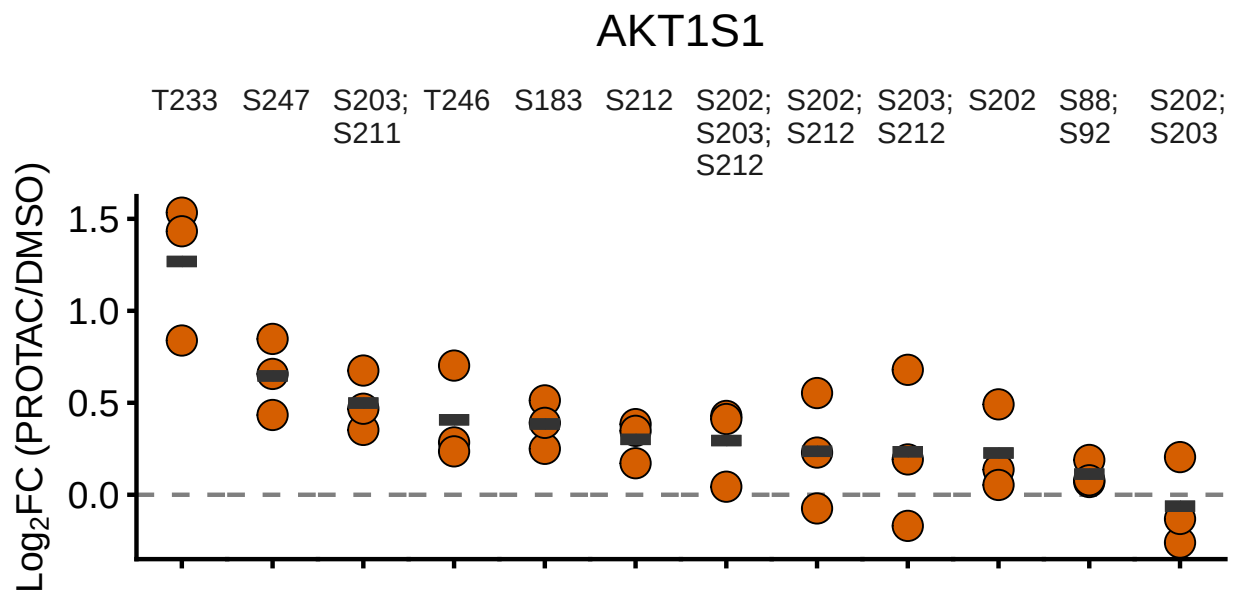
```
ggsave(plot=p1, here('results/plots/PROTAC_AKT1S1_phosphosite_changes_selected.png'))
```

Saving 6.5 x 4.5 in image

```
ggsave(plot=p1, here('results/plots/PROTAC_AKT1S1_phosphosite_changes_selected.pdf'))
```

Saving 6.5 x 4.5 in image

```
p2 <- p + filter(to_plot, condition!='DMSO')
print(p2)
```



```
ggsave(plot=p2, here('results/plots/PROTAC_AKT1S1_phosphosite_changes.png'))
```

```
## Saving 6.5 x 4.5 in image
```

```
ggsave(plot=p2, here('results/plots/PROTAC_AKT1S1_phosphosite_changes.pdf'))
```

```
## Saving 6.5 x 4.5 in image
```

Save results

```
phospho_limma_results %>%
  select(UniprotID, phosphosite, phospho_pos, logFC, P.Value, adj.P.Val,
         Protein.names=Protein.names.y, Gene.Names, Gene.Names.First, site_seq) %>%
  write.table(here('results/PROTAC_phospho_limma_results.tsv'),
             sep='\t', row.names=FALSE, quote=FALSE)
```

Finally, session info:

```
sessionInfo()
```

```

## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Sonoma 14.7
##
## Matrix products: default
## BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK v
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/London
## tzcode source: internal
##
## attached base packages:
## [1] stats4 stats graphics grDevices utils datasets methods
## [8] base
##
## other attached packages:
## [1] ggbeeswarm_0.7.2 tidyr_1.3.1
## [3] dplyr_1.1.4 ggrepel_0.9.6
## [5] ggplot2_4.0.0 limma_3.64.3
## [7] here_1.0.2 biomasslmb_0.0.4
## [9] QFeatures_1.18.0 MultiAssayExperiment_1.34.0
## [11] SummarizedExperiment_1.38.1 Biobase_2.68.0
## [13] GenomicRanges_1.60.0 GenomeInfoDb_1.44.2
## [15] IRanges_2.42.0 S4Vectors_0.48.0
## [17] BiocGenerics_0.54.0 generics_0.1.4
## [19] MatrixGenerics_1.20.0 matrixStats_1.5.0
##
## loaded via a namespace (and not attached):
## [1] DBI_1.2.3 rlang_1.1.6 magrittr_2.0.4
## [4] clue_0.3-66 compiler_4.5.2 RSQLite_2.4.3
## [7] systemfonts_1.3.1 png_0.1-8 vctrs_0.6.5
## [10] reshape2_1.4.4 stringr_1.5.2 ProtGenerics_1.40.0
## [13] pkgconfig_2.0.3 crayon_1.5.3 fastmap_1.2.0
## [16] backports_1.5.0 XVector_0.48.0 labeling_0.4.3
## [19] rmarkdown_2.29 UCSC.utils_1.4.0 ragg_1.5.0
## [22] visdat_0.6.0 purrr_1.1.0 bit_4.6.0
## [25] xfun_0.54 cachem_1.1.0 jsonlite_2.0.0
## [28] blob_1.2.4 DelayedArray_0.34.1 cluster_2.1.8.1
## [31] R6_2.6.1 stringi_1.8.7 RColorBrewer_1.1-3
## [34] genefilter_1.90.0 Rcpp_1.1.0 knitr_1.50
## [37] BiocBaseUtils_1.10.0 Matrix_1.7-4 splines_4.5.2
## [40] igraph_2.1.4 tidyselect_1.2.1 rstudioapi_0.17.1
## [43] abind_1.4-8 yaml_2.3.10 lattice_0.22-7
## [46] tibble_3.3.0 plyr_1.8.9 withr_3.0.2
## [49] KEGGREST_1.48.1 S7_0.2.0 evaluate_1.0.5
## [52] uniprotREST_1.0.0 survival_3.8-3 Biostrings_2.76.0
## [55] pillar_1.11.1 corrplot_0.95 checkmate_2.3.3
## [58] rprojroot_2.1.1 scales_1.4.0 xtable_1.8-4
## [61] glue_1.8.0 lazyeval_0.2.2 tools_4.5.2
## [64] robustbase_0.99-6 annotate_1.86.1 XML_3.99-0.19
## [67] grid_4.5.2 MsCoreUtils_1.20.0 AnnotationDbi_1.70.0

```

```

## [70] GenomeInfoDbData_1.2.14 beeswarm_0.4.0      vipor_0.4.7
## [73] naniar_1.1.0          cli_3.6.5      textshaping_1.0.3
## [76] S4Arrays_1.8.1        AnnotationFilter_1.32.0 gtable_0.3.6
## [79] DEoptimR_1.1-4        digest_0.6.37   SparseArray_1.8.1
## [82] farver_2.1.2          memoise_2.0.1   htmltools_0.5.8.1
## [85] lifecycle_1.0.4       httr_1.4.7      statmod_1.5.0
## [88] bit64_4.6.0-1         MASS_7.3-65

```